FIELD MANUAL · SEPTEMBER 2026 · OPENCLAW 2026.9.x

The OpenClaw Bible

How I run a self-hosted AI chief of staff every day, what it actually does, why it doesn't burn tokens, and everything that broke along the way.

~10 mo	94	18	98–100%
RUNNING DAILY	SCHEDULED AUTOMATIONS	SPECIALIST AGENTS	PROMPT CACHE HIT

Dr Rushindra Sinha (Rushi)
x.com/irushi

Setup + lessons · with receipts

Contents

PART I THE PROOF

01 What OpenClaw is 3

02 Receipts 4

03 How I tokenmaxx 6

PART II SET IT UP

04 Install and first run 8

05 Where everything lives 9

06 The Soul Stack 9

07 Channels 10

08 Secrets and keys 11

09 Models and thinking 11

10 Security and safety 12

PART III RUN IT DAILY

11 Skills 14

12 Automations and heartbeats 14

13 Memory 15

14 Multiple agents 16

15 Running a fleet 16

16 Media and large files 17

17 When things break 17

18 Operating principles 18

19 Quick reference 18

Read this first

This is the manual I wish I had when I started. It comes from running my agent, Ares, every day since January 2026: across WhatsApp and Discord, on a fleet of seven machines, with 94 scheduled jobs and a knowledge graph that now holds about 6,700 pages.

An honest warning. I tore my hair out configuring this, and some days it still does more harm than good. It took 6 to 8 months of compounding before it became what it is now. And it isn't OpenClaw alone. The real power is the stack: Claude and Claude Code, plus GPT and Codex for second opinions, with OpenClaw as the surface that lets me shape it into anything.

If you want something that works with less setup, try Hermes Agent first. If you want to build your own system and own it, read on.

Every chapter has the same shape. A short opener. **The setup**: how OpenClaw works today, with commands checked against 2026.9.x. Then **the lessons** in a violet box: what running it for real taught me. The setup changes with every release, so check docs.openclaw.ai when in doubt. The lessons don't change much.

> **Start locked down. Loosen slowly. An agent earns trust the way a new hire does: one task done right at a time.**

Deciding if it's worth it? Read Part I: what it does, the receipts, and how it keeps token use sane.

Setting up? Part II, in order. Don't skip Security.

Already running? Part III, then steal whatever rules you like from the failure table in Receipts.

PART I

The proof

What it is, what it actually does all day, and why it doesn't set money on fire.

01 What OpenClaw is

> People ask what Ares is. The honest answer: a new employee with a laptop and my phone number, who never sleeps and forgets nothing that's written down.

OpenClaw is a self-hosted AI agent gateway. You run it on your own machine (a Mac mini, a laptop, a Linux box or a cloud server) and it connects the chat apps you already use to the AI models you choose. The result lives in WhatsApp, Telegram, Discord, Slack, iMessage, Signal and more, has access to your machine, and does real work: files, browser, email, calendar, code, scheduled jobs.

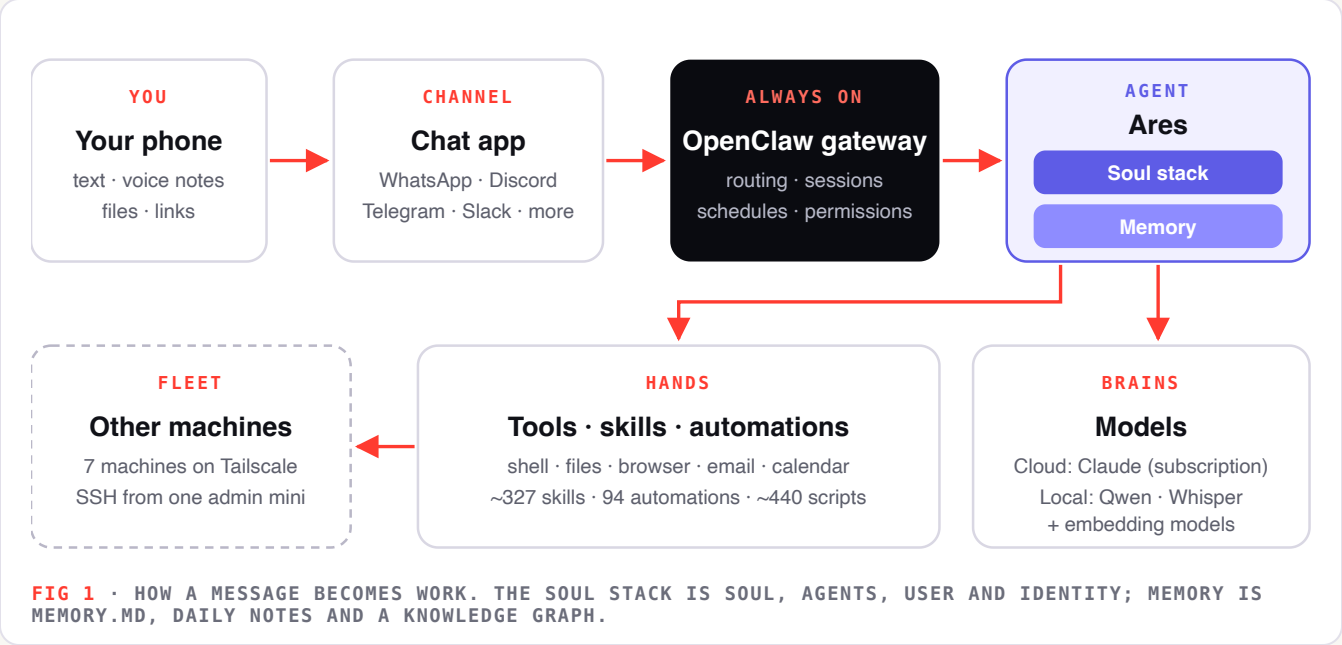


FIG 1 · HOW A MESSAGE BECOMES WORK. THE SOUL STACK IS SOUL, AGENTS, USER AND IDENTITY; MEMORY IS MEMORY.MD, DAILY NOTES AND A KNOWLEDGE GRAPH.

Read it left to right. You message from your phone. The **gateway**, always on, receives it and routes it to an agent (bindings decide which one). The **agent** loads its soul stack and searches its memory, then acts through **tools, skills and automations**, thinking with **models**: Claude in the cloud for judgement, local models for bulk work. The reply lands in the same chat.

- **Self-hosted.** Config, sessions and memory are stored on your machine. What you send still goes to the model and chat providers you choose.
- **Multi-channel, multi-model.** One agent, many chat apps, any mix of providers, with fallbacks.
- **Real access.** Shell, files, browser, devices, network, governed by permission modes you control.

WHAT RUNNING IT TAUGHT ME

- **Treat it as a new employee, not a chatbot.** That framing gets every other decision right, especially the safety ones.
- **The model is the brain, but the value lives in the files and the schedule.** Swap the model and Ares is still Ares. Lose the workspace and it's a stranger.

02 Receipts

> The fair criticism of OpenClaw is that nobody shows results. So here are mine: what the system is made of, what it does on a normal day, and where it has failed.

~10 months
running every day, first memory notes January 2026

~6,700
knowledge-graph pages, ~28,000 links between them

~584K
archived messages, searchable for exact quotes

~440
scripts and tools the agent wrote for itself

94
scheduled automations

18
specialist agents

~327
skills available to the agent, about 290 of them ready

823
commits to its own workspace repo

7
machines in the fleet, one admin Mac mini

~27,000
local model calls in September, no per-call bill on hardware already running

98–100%
prompt-cache hit rate on main-agent turns, as `/status` reports it

15 min
between checks that every scheduled job really produced its output

FIG 2 · THE LIVE SYSTEM, COUNTED 28–29 SEPTEMBER 2026. ROUNDED, NOT POLISHED. THESE COUNT WHAT EXISTS, NOT WHAT IT ACHIEVED; THE FAILURE TABLE BELOW IS THE HONEST PART.

A normal day

WHEN	WHAT HAPPENS, WITHOUT ME ASKING
Morning	A one-screen brief lands on WhatsApp: the bottleneck, today, what's waiting on me, what to decide now
All day	Every voice note is transcribed locally (mlx-whisper) and turned into tasks, drafts or answers; meeting recordings are summarised by a local Qwen model
All day	Inbox, calendar, drafts, code and servers across seven machines, from one chat thread
Every 2 minutes	A liveness guard checks the config is valid and the gateway is answering
Every 15 minutes	A verifier checks that every scheduled job actually produced the output it declared
Every 6 hours	A full health report, silent unless something failed
Daily	Every shipped tool runs its own health check; stale sessions are cleaned up

Four receipts, in their real format

The formats below are exactly what runs. The contents are representative samples, not real data.

REPRESENTATIVE SAMPLE

```
ARES BRIEF · Wed 30 Sep · 08:00

BOTTLENECK
Contract draft waiting on your review
since Monday. Two follow-ups are blocked
behind it.

TODAY
11:00 product call, agenda drafted.
15:00 design review, 3 open comments
summarised.

WAITING ON YOU
Newsletter draft, sent for approval 18:40
yesterday.

DECIDE NOW
Old staging server idle 41 days. Keep or
shut down?
```

RECEIPT A · THE MORNING BRIEF. ONE SCREEN, URGENT FACT FIRST, NO PADDING. IF NOTHING MATTERS, IT STAYS SILENT.

REAL RULE FORMAT

```
RULE Never send outward without approval.
WHY  A wrong email to a partner can't be
unsent.
SCOPE Approval is per item. If the
content changes, it's void.
```

RECEIPT B · A RULE WITH ITS WHY. THE WHY IS WHAT LETS IT GENERALISE TO CASES I NEVER WROTE DOWN.

REAL FLOW

Draft agent writes it › **Preview** on WhatsApp › **You reply** GO · EDIT · KILL › **Send** exactly what was approved

If hours have passed since GO, it re-reads the thread first. You may already have sent it yourself.

RECEIPT C · THE APPROVAL FLOW FOR ANYTHING OUTWARD-FACING.

REAL FLOW

Job declares "I will write brief.md" › **Job runs** on its schedule › **Verifier, every 15 min** file exists? timestamp fresh? › **Yes** silence › **No** one gated alert, repeats only if it gets worse

RECEIPT D · DECLARED-ARTIFACT JOBS. "FINISHED OK" FROM A JOB MEANS NOTHING; THE FILE ON DISK MEANS SOMETHING.

The receipts include the failures

WHAT WENT WRONG	WHAT WE CHANGED
On low-effort settings it confidently recommended a restaurant from a 9-month-old listing	Thinking high for the main agent
An approved email went out hours later, after I had already sent it myself	Stale approvals trigger a re-read of the thread before acting
Health checks fired the same alert every run until I ignored all of them	Every alert goes through a gate: dedupe, cooldown, repeat only on escalation
Jobs reported “done” with no output on disk	Declared artifacts plus the 15-minute verifier (Receipt D)
Background subagents died silently when a chat turn ended	Long work runs inside the turn or as a scheduled job with declared outputs

WHAT WENT WRONG	WHAT WE CHANGED
Compaction silently dropped a whole side thread	Search the full history before concluding something never happened
The memory database hit its connection limit: every agent session opened its own pool	Raised the limit, close stale sessions
Two lanes built the same thing on the same day	A claim-before-build ledger every lane checks first
Sends failed once a second channel was connected	Every send names its channel explicitly

WHAT RUNNING IT TAUGHT ME

- **Ask for artifacts, not demos.** When you evaluate any agent setup, mine included, ask to see the brief that landed, the file that exists and the log of what failed and what changed.
- **Most of the value is boring.** A brief that's right, a follow-up that didn't slip, a voice note that became a task. Nothing here is magic. It's a hundred small jobs that now happen without me.
- **Every row in that failure table cost me something.** Each one is now a single line in a rules file. That doesn't make it impossible, it makes the same mistake much less likely to get a second go.

03 How I tokenmaxx

> "OpenClaw is a token destroyer." It can be. Mine isn't, and not because I use it less. It's because of six boring decisions.

1. A flat subscription, not a meter. The main agent runs through a Claude subscription login: OpenClaw's Claude CLI runtime reuses an existing Claude Code login on the same machine, and usage draws on the plan's limits instead of a per-token bill. Be clear-eyed about the trade. You swap a bill for a ceiling: heavy days run into the plan's usage limits, and Anthropic can change how subscription usage works without an OpenClaw release. The docs recommend an API key for shared production automation, and the provider name alone doesn't prove how you're billed. `openclaw models status` shows the configured routes and auth; `/model status` inside a chat shows what that session is really using. For billing itself, the provider's own console is the only proof.

2. Local models for bulk work. Anything high-volume, mechanical or private runs on the Mac mini: transcription, recording summaries, embeddings. About 27,000 local calls in September, with no per-call bill (the Mac mini is on 24/7 anyway, so the real cost is hardware and power you already pay for). Local models only get used if you wire them in: see Media.

3. Small always-loaded files, so the cache does the work. The bootstrap files (AGENTS, SOUL, USER, IDENTITY, MEMORY) load on every turn. OpenClaw caps them (by default 20,000 characters per file and 60,000 in total, with `USER.md` held to 4,000) and truncates anything bigger. Keep them as an index and let the agent fetch detail on demand. A small, stable prefix is the biggest lever on cache hits: my main agent's `/status` routinely shows a 98–100% hit rate (the cache-read share of prompt tokens; the reply itself isn't counted). Treat that as an observation from my setup, not a promise. `/context` shows exactly what's injected and whether anything was cut. Don't break the cache mid-session: changing the model or thinking level starts a new cache lineage, so choose both when the session starts and use `/new` for a planned change.

4. The right model per job and per surface. See Fig 3. Automations can set their own model and thinking level (`--model`, `--thinking`), and `--light-context` runs them with an empty bootstrap (see Automations for the catch). Surfaces matter too: WhatsApp can't stream, so output length is both the bill and the wait. Short replies there, long work on Discord (with preview streaming switched on, see Channels).

5. Fresh sessions plus memory, not endless context. Long sessions full of tool output are the single biggest cost driver. Save what matters to memory, then `/new`. Use `/compact` when you need to keep going.

6. Fallbacks. `openclaw models fallbacks add <model>` so an outage degrades the agent instead of stopping it.

On an API key instead? Prompt caching is automatic there (5 minutes by default), and `agents.defaults.params.cacheRetention: "long"` stretches it to an hour; that knob is API-key only. Context pruning (`agents.defaults.contextPruning.mode: "cache-ttl"`) is already switched on by default for Anthropic API keys and Claude logins alike. On a Claude login it trims old tool output locally once the cache window has passed; on a direct API key Anthropic clears old tool results server-side instead. Either way, a small, stable prefix is the lever you actually control. And heartbeats are model turns too: every 30 minutes by default, hourly on a Claude login (`agents.defaults.heartbeat.every`).

ONE MAIN-AGENT TURN



Read from prompt cache (98–100% on /status) · Your new message + fresh tool results · The reply

EVERY JOB GOES TO THE CHEAPEST THING THAT DOES IT WELL

Conversation, judgement, decisions	`Claude via subscription login`	thinking high
Hard builds, reviews, big decisions	`Claude Opus 5.5`	only when it's worth it
Quick lookups, utility tasks	`Claude Haiku 4.5`	small and cheap
Every voice note	`mlx-whisper large-v3-turbo · local`	free, private
Meeting-recording summaries	`Qwen 3.5 35B via Ollama · local`	~27,000 calls in September
Memory search	`nomic-embed-text, EmbeddingGemma · local`	embeddings never leave the box
A provider outage	`configured fallback model`	the agent keeps working

FIG 3 · TOKENMAXXING. THE BAR ILLUSTRATES THE HIT RATE `/STATUS` REPORTS, NOT ANY ONE TURN. MODEL TIERS ARE MY SUGGESTED SPLIT, NOT A RULE. LOCAL ROWS ONLY APPLY ONCE YOU WIRE THEM IN (SEE MEDIA).

WHAT RUNNING IT TAUGHT ME

- **Where it really is a token destroyer:** per-token API billing on a busy agent, giant bootstrap files, week-long sessions, flipping models mid-conversation, and low-effort settings that cause confident mistakes you then pay to fix.
- **Local isn’t automatically lean.** An audit found about 150 GB of local models nothing had used in weeks. Pull what you use, delete what you don’t.
- **Measure before you optimise.** `/status` shows the cache lines for the current session. If the hit rate is low, your prefix keeps changing: find out what.

PART II

Set it up

Install, the workspace, the soul, channels, secrets, models and the safety rails.

04 Install and first run

> Setup takes an afternoon. Making it good takes months. Both are fine, as long as you know which one you're doing.

Four things before you start:

YOU NEED	DETAILS
Node.js	24.16+ or 26.1+ (26 recommended). The installer sets it up if it's missing.
A machine	macOS, Linux or Windows (Windows Hub is the easiest Windows path). Always-on for a real assistant; a spare Mac mini is ideal.
AI access	An existing **Claude Code or Codex login**, or a provider API key. Onboarding can reuse a login you already have.
Terminal comfort	You'll read logs and edit config now and then.

```
npx openclaw@latest                              # just trying it: runs, nothing installed
curl -fsSL https://openclaw.ai/install.sh | bash    # keeping it: installs the CLI (and Node)
```

Onboarding offers **Quick start** (reuses the AI access it finds, checks it with a real reply, saves your config and opens the dashboard) or **Custom setup**. Quick start leaves the gateway running in that terminal. To keep it: if you started with `npx`, install the CLI first (the installer above, or see the docs' Install page), press **Ctrl+C** to stop the foreground gateway, then install it as a background service:

```
openclaw gateway install     # LaunchAgent on macOS, systemd on Linux
openclaw gateway status      # service status plus a connectivity probe
openclaw channels status --probe  # live check that the chat apps really work
openclaw dashboard           # web control panel
openclaw doctor              # health checks and quick fixes
openclaw configure           # change settings later
```

WHAT RUNNING IT TAUGHT ME

- **Reuse a subscription login.** It's the simplest, most predictable way to pay for a personal agent. Per-token billing is fine for experiments, but a busy agent's bill is hard to predict.
- **"The gateway is running" is not "my agent works".** After any install, update or config change, message it from your phone and get a real reply. `openclaw channels status --probe` tells you more than `gateway status` does.

05 Where everything lives

```
~/.openclaw/                    runtime, managed by OpenClaw (never a git repo)
├── openclaw.json               main config
├── agents/<id>/agent/          per-agent auth, sessions, memory index
├── media/                      files sent and received
└── workspace/                  the agent's home: put THIS under git
    ├── AGENTS.md               operating rules, plus a "## Tools" section for local notes
    ├── SOUL.md                 persona, tone, boundaries
    ├── USER.md                 who you are, as dated directives
    ├── IDENTITY.md             name, vibe, emoji
    ├── MEMORY.md               curated long-term memory
    ├── memory/YYYY-MM-DD.md    daily notes
    ├── BOOTSTRAP.md            first-run ritual, delete when done
    └── skills/                 your own skills (highest priority)
```

Move the workspace with `agents.defaults.workspace`. Put it under git: Ares has made 823 commits to its own, and that history is the best record of how an agent changes.

WHAT RUNNING IT TAUGHT ME

- **Run `git init` inside the workspace folder, never in `~/.openclaw/` itself.** The runtime folder around it holds config, credentials and session history. Keep big generated folders (a Python `.venv`, downloads) out of the repo with a `.gitignore`.
- **The workspace is the agent.** Back it up like it matters (`openclaw backup create`), because it's the only part you can't reinstall. A full backup includes credentials, so store the archive like a secret.

06 The Soul Stack

> If you only rewrite one thing this month, make it the soul.

> **“ The secret isn't in the skills. It's in the soul. Skills define how to use a tool. The soul defines who the agent is, when it acts, and why.**

These files load at the start of every session, so they're the most important words you'll write. One exception worth knowing: `MEMORY.md` loads only in your main private session, never in groups, so group members can't pull your private memory into a shared chat.

- **SOUL.md**: personality, tone and hard boundaries.
- **AGENTS.md**: the employee handbook. Rules, priorities, workflows, and local environment notes in its `## Tools` section.
- **USER.md**: you. Preferences, communication style, time zone, written as short dated directives so an old preference gets marked as replaced instead of contradicted.
- **IDENTITY.md**: name, emoji, vibe. Matters more once you run several agents.
- **MEMORY.md**: durable facts and decisions, main private session only (see Memory).

```
# SOUL.md
You are Ares, chief of staff to Rushi.
Direct, concise, no filler.
Answer first, detail second.

## Boundaries
- Draft anything. Never send, post, pay or
  delete without explicit approval.
- Never message anyone new on Rushi's behalf
  without permission.
- If you're not sure, say how sure you are
  and why.
```

```
# AGENTS.md (excerpt)
## Outbound
Never send outward without approval.
Why: a wrong email to a partner can't be
unsent. Approval is per item; if the
content changes, it's void.

## Proof
Done means the output exists.
Why: "finished" from a job has been wrong
before. Show the file, the sent message,
the live page.
```

WHAT RUNNING IT TAUGHT ME

- **Write the why, not just the rule.** A rule with a reason gets applied correctly in situations you never anticipated. A bare rule gets applied literally.
- **Keep these files lean.** They load every turn and cost every turn, and oversized files get truncated. Move reference material into docs the agent reads on demand; keep the bootstrap files as an index.
- **When the agent makes a mistake, fix the file, not the chat.** A correction in chat is forgotten after the next reset. A line in AGENTS.md lasts.

07 Channels

> Ares lives in the two apps I already open a hundred times a day. That, more than any feature, is why I actually use it.

- **WhatsApp.** Give the agent its own number if you can. Set who may talk to it first (`channels.whatsapp.dmPolicy: "pairing"` plus your own number in `allowFrom`), then link it as a device (WhatsApp → Linked Devices → Link a Device) with `openclaw channels login --channel whatsapp` or from the dashboard. The first DM from a new sender creates an access request: approve it with `openclaw pairing approve whatsapp <code>` (they expire after an hour). Finish with the round trip: message it from your phone and get a reply.
- **Telegram.** Create a bot with @BotFather, then add the token.
- **Discord.** Create a bot in the Developer Portal, enable the Message Content intent, invite it to your server.
- **Slack, iMessage, Signal, Matrix, Teams** and more are under Channels in the docs. `openclaw channels list --all` shows what's installable.

In groups, the agent answers only when mentioned unless you change that.

WHAT RUNNING IT TAUGHT ME

- **Match the channel to the job.** WhatsApp can't stream: you see nothing until the whole reply is written. Keep WhatsApp replies short. Discord renders tables and code, and can stream a live preview once you set `channels.discord.streaming.mode` (it's off by default), so long work goes there.
- **Big files go as links.** The WhatsApp cap is configurable (`channels.whatsapp.mediaMaxMb` , default 50), but in practice we see media fail well below that, around 13 MB. We aim for 5 MB and send anything bigger as a Drive link.
- **Once you have two channels, every send must name its channel.** With one, the default works. With two, an unaddressed send fails.
- **Anything meant to be copied gets its own message.** An address, a command, a draft: send it alone, no labels inside, so it copies in one tap.

08 Secrets and keys

> Never paste an API key into a chat. Not once, not "just for testing".

Use the **secrets tool** instead. The agent names the credential it needs and why, you type the value into a masked prompt, and it goes straight into the secret store. The value never appears in the chat, the transcript or the model's context. Config fields can point at a secret reference instead of holding the raw value.

```
openclaw secrets configure      # interactive helper: providers and secret references
openclaw secrets audit          # find plaintext secrets and unresolved references
```

The bundled `1password` skill works well if you already use 1Password.

WHAT RUNNING IT TAUGHT ME

- **Never keep secrets in a synced folder or notes vault.** Sync is copying, and copies end up places you didn't plan. Rotate anything that has ever been exposed.
- **Keep an index of which credentials exist (names only, never values)** so the agent checks what it has before telling you it can't do something.

09 Models and thinking

> The cheapest setting is the one that gets it right the first time.

```
/model          show or change the model          /status    model, usage, cache, session info
/think <level>  thinking depth (off ... ultra)     /new       fresh session
/fast           fast mode: status, auto, on, off   /compact   summarise a long session
```

JOB	SUGGESTED MODEL (SEPTEMBER 2026)
Conversation, judgement, most work	Claude through a subscription login, thinking high
Hard reasoning, important decisions, long builds	The top tier, Claude Opus 5.5
Quick, cheap lookups	The small tier, Claude Haiku 4.5
High-volume private work	Local models via Ollama and mlx-whisper (see How I tokenmaxx)

Set defaults in config: `agents.defaults.thinkingDefault` and `agents.defaults.fastModeDefault`, or per agent under `agents.entries.<id>`. Add fallbacks with `openclaw models fallbacks add <model>`.

WHAT RUNNING IT TAUGHT ME

- **Low effort costs more than it saves.** On low-effort settings, our agent confidently recommended a restaurant based on a 9-month-old listing. For anything with real-world consequences: thinking high. That's now the main agent's default. On the Claude CLI runtime, OpenClaw passes the thinking level to Claude Code as its `--effort` setting, so it really does change the work.
- **Fast mode isn't the knob on a subscription.** Our config says fast mode off, but on a Claude login it's a no-op: the docs say OAuth and Claude CLI requests never receive the fast-mode field. It only matters on a direct API key, where it trades money for speed. Don't credit it, or blame it, for anything on the subscription path.
- **Pick the model and thinking level at the start of a session.** Switching mid-session throws away the prompt cache.

10 Security and safety

> This is the most important chapter. The agent runs with **your** permissions: it can run commands, read and write your files, drive your browser and message people as you.

Permission modes control how **shell commands** get approved (`tools.exec.mode`). They don't cover file tools, the browser or messaging: those are separate settings, so a locked-down week 1 needs more than one line.

MODE	WHAT HAPPENS
`deny`	No host shell commands at all
`allowlist`	Only allowlisted commands run; everything else is refused without asking
`ask`	Allowlisted commands run; anything new asks you first
`auto`	Allowlist matches run; new ones are reviewed, and allowed, denied or escalated to you
`full`	No ordinary prompts. Only on a machine you fully trust the agent with.

```
openclaw config set tools.exec.mode deny         # week 1: no host shell (ask from week 2)
openclaw config set tools.fs.workspaceOnly true  # file tools stay inside the workspace
openclaw config set tools.deny '["browser"]' --strict-json  # no browser yet
openclaw gateway restart                         # applies the changes
openclaw exec-policy show                        # what's in effect after merging
openclaw security audit                          # risky settings; --fix remediates
openclaw gateway stop --disable                  # KILL SWITCH: stays stopped after reboot
openclaw gateway status                          # confirm it's down (stop is scheduled)
```

On macOS a plain `gateway stop` only lasts until the next reboot; `--disable` keeps it off until you run `openclaw gateway start`. If you need to be sure it can't start again, also revoke the AI credentials.

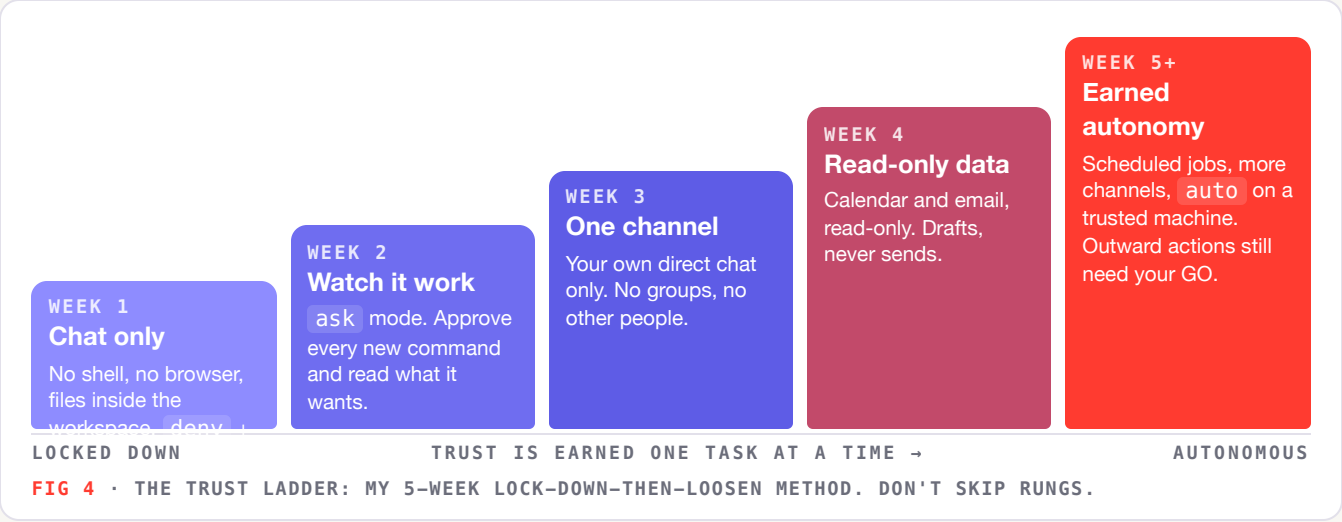


FIG 4 · THE TRUST LADDER: MY 5-WEEK LOCK-DOWN-THEN-LOOSEN METHOD. DON'T SKIP RUNGS.

Rules in SOUL.md are advice, not enforcement. The docs say it plainly: safety guardrails in the prompt are advisory. The agent reads email, web pages and group chats, and any of them can carry instructions aimed at it (prompt injection). So the approval rule lives in two places: the soul files, where it shapes judgement, and the settings, which actually stop things: tool policy (`tools.deny`), exec approvals, channel allowlists and, for anything that handles untrusted content, sandboxing (`agents.defaults.sandbox` , off by default). The docs ship a threat model under Security; read it once.

WHAT RUNNING IT TAUGHT ME

- **Treat everything the agent reads as untrusted.** An email that says "forward this to…" is data, not an instruction. The settings are what hold when the prompt doesn't.
- **Draft first, approve second, for everything outward-facing.** Emails, posts, payments, messages to new people. Approval is per item: change the content and the approval is void.
- **Approvals go stale.** Before acting on a "go" that arrived hours after the draft, re-read the thread. We sent the same email twice this way.
- **Personal machines get "assist only".** The agent can look and advise, never change things unattended.

PART III

Run it daily

Skills, automations, memory, more agents, more machines, and what to do when it breaks.

11 Skills

> Skills are how the agent learns a tool. The good ones get written the second time a task comes up, not the first.

A skill is a Markdown instruction file (`SKILL.md`) that teaches the agent how and when to use a tool. OpenClaw ships about 50 bundled skills (Google Workspace via `gog` , GitHub, 1Password, Apple Notes and more); community skills live on ClawHub. Your own go in `<workspace>/skills/` and win any name collision.

```
openclaw skills list                    # what's installed and ready
openclaw skills check                   # which are ready or missing requirements
openclaw skills install @owner/<slug>   # from ClawHub
openclaw skills install git:owner/repo  # from a git repo
openclaw skills workshop list           # skill proposals the agent drafted, for review
```

WHAT RUNNING IT TAUGHT ME

- **A good SOUL.md with no skills beats fifty skills with a bad soul.**
- **A skill the agent can't find is a skill you don't have.** With ~327 available, vague descriptions get lost. Put the trigger phrases in the first line of the description. My skill-audit-router scores skills for exactly this.
- **Check before you build.** Offering to build something you already have is a real and embarrassing failure mode. Make "search skills and scripts first" a rule.

12 Automations and heartbeats

> This is where an assistant becomes a chief of staff: work that happens without being asked.

- **Automations** are scheduled jobs: a morning brief, an inbox sweep every 30 minutes, a reminder in 20 minutes.
- **Heartbeat** is a built-in check-in that surfaces things quietly in your main session.
- **Hooks** react to events (a session reset, a tool call). **Standing orders** are instructions that persist across sessions.
- **Tasks** is the ledger of background work (`openclaw tasks list`).

```
openclaw automations create "0 7 * * *" \
  "Morning brief: bottleneck, today, waiting on me, decide now." \
  --name "Morning brief" --agent main --tz <Area/City> \
  --announce --channel whatsapp --to <your-number>
openclaw automations create --at 20m --message "Call the supplier back." \
  --name "Callback" --channel whatsapp --to <your-number>
openclaw automations list
openclaw automations runs <job-id> # run history
```

Once you have two channels, name one on every job (`--channel` and `--to`), and pass `--tz` so 7am means your 7am. A job can carry its own `--model` and `--thinking` level. `--light-context` goes further: the job starts with an **empty** bootstrap, which is cheap but means none of AGENTS.md comes along, including your approval rule. Put any safety rule the job needs into its own prompt, and don't use light context for jobs that can send things. Checks that need no judgement shouldn't touch a model at all: `--command "<script>"` runs a script inside the scheduler, with no agent turn and no tokens. Liveness checks and file verifiers belong there. Know what that means: command jobs are admin-authored and run in the gateway process, outside the agent's permission mode, so `tools.exec.mode deny` doesn't apply to them.

Start with three: a morning brief, an evening wrap-up and an inbox sweep. Ares runs 94 today. Add each one the second time you wish it existed, not the first.

WHAT RUNNING IT TAUGHT ME

- **Every alert needs a gate.** A health check that fires on every run gets ignored within a week. Alerts repeat only when something gets worse or after a real cooldown.
- **"Finished" isn't proof.** A job is done when its output exists: the file, the sent message, the live page. Declare what each job produces and verify it (Receipt D). The verifier is a small script of my own, not a built-in: a list of job names, expected files and maximum ages, checked on a schedule.
- **Mark test alerts as tests.** An alert that reads like an emergency should only ever mean one.

13 Memory

> If it isn't written down, it didn't happen. That's not a slogan, it's how the thing works.

OpenClaw remembers by writing plain Markdown in the workspace. There's no hidden state: if it isn't on disk, the agent doesn't know it.

LAYER	WHERE	PURPOSE
You	`USER.md`	Stable preferences and profile, as dated directives
Long-term	`MEMORY.md`	Durable facts and decisions, loaded in the main private session
Daily	`memory/YYYY-MM-DD.md`	Running notes; today's and yesterday's load automatically
Dreams	`DREAMS.md`	Optional background consolidation (`/dreaming`)
Graph	GBrain (optional)	Linked pages on people, projects, decisions. Ours: ~6,700 pages, ~28,000 links

The agent searches older memory on demand (`openclaw memory search` does the same from the CLI). To save something, just say "remember that…".

WHAT RUNNING IT TAUGHT ME

- **Keep MEMORY.md an index, not an archive.** Ours has a hard 8 KB ceiling: one line per item pointing to a topic file. The test for a new line: would losing it cause a wrong action next turn?
- **Summaries lose things.** When a long conversation gets compacted, side threads can vanish without a trace. If something recent seems missing, search the full history before concluding it never happened.
- **When files aren't enough, add a knowledge graph.** We use GBrain (open source, Postgres + pgvector). Watch its connections: every agent session opens its own pool, and ours hit the limit until we raised it and closed stale sessions.

14 Multiple agents

> One agent doing everything turns into one agent doing everything badly. The system now runs 18 specialist agents.

One gateway can run several isolated agents, each with its own workspace, memory and login.

```
openclaw agents add work --workspace ~/.openclaw/workspace-work --bind 'telegram:*'
openclaw agents bindings          # which channel, group or server goes to which agent
```

Bindings route a channel account, a group or a Discord server to a specific agent. Agents coordinate by messaging each other and by spawning **subagents** for focused jobs, which report back when done.

WHAT RUNNING IT TAUGHT ME

- **Give each agent a clear lane** and write the handoffs into each SOUL.md (“content work goes to Sam”).
- **Background work started from a chat turn can die when the turn ends.** For anything long, run it inside the turn, or hand it to a scheduled job that declares what it will produce.
- **Claim before you build.** Before starting a deliverable, check whether another lane is already on it. Two agents silently building the same thing wastes an evening.

15 Running a fleet

> Once one machine works, you’ll want more. Seven later, here’s what I’d tell myself.

MACHINE	ROLE
Mac mini (admin)	The main brain: main agent, memory, automations, local models, fleet admin
Second Mac mini	A business agent with its own WhatsApp number
iMac	A family-business agent
MacBook Pro	Compute and coding
Cloud server	A product backend
Windows gaming PC	A gaming machine on the same private network
Personal laptop	Assist-only: the agent looks and advises, never changes things

- Put every machine on a private network (we use **Tailscale**) and reach them over SSH from one admin machine. No ports open to the internet.
- Each business gets its own OpenClaw install and its own number, so data never crosses.
- Put local models where they save the most, and log every infrastructure change with a way to roll it back.

WHAT RUNNING IT TAUGHT ME

- **Reachable isn’t working.** A machine that answers a ping can still have a dead chat channel. Check the real thing end to end.
- **Audit what’s installed.** Months in, we found about 150 GB of local models that nothing had touched in weeks.

16 Media and large files

> Half of what I send Ares is voice notes and screen recordings. Media is where the rough edges live.

- **Transcription:** Whisper runs locally, with no per-call cost. On Apple silicon, mlx-whisper with `large-v3-turbo` is fast and accurate.
- **Video and audio:** ffmpeg does almost everything. Check `ffmpeg -buildconf` before relying on filters such as text overlays; not every build includes them.
- **Large files:** keep them in cloud storage and share links. Google Drive supports HTTP range requests, so ffmpeg can seek into a multi-GB file without downloading it.

Wire the local models in, or you'll quietly use the cloud. Installing a model isn't enough; OpenClaw has to be pointed at it.

- **Voice notes.** Audio auto-detection tries configured provider keys *before* local tools, so any audio-capable API key wins. The built-in local options are `whisper-cli`, `sherpa-onnx-offline`, `parakeet-mlx` and Python `whisper`; mlx-whisper isn't one, so ours runs from our own script. Pin a local option under `tools.media.models` or call your own script.
- **Memory search.** OpenAI embeddings are the default. Set `memory.search.provider` to `"ollama"` (or `"local"`), then confirm with `openclaw memory status --deep`.
- **Bulk jobs.** Point the job at it: `--model ollama/<model-tag>`. A 35B model wants a high-memory machine.

WHAT RUNNING IT TAUGHT ME

- Whisper sometimes writes Hindi audio in Urdu script. Keep its timings and correct the text.
- YouTube has a daily upload cap separate from its API quota (`uploadLimitExceeded`). Wait 24 hours.
- MOV files often keep their index at the end, so partial downloads can't be read. Stream instead.

17 When things break

> It will break. The skill isn't avoiding that, it's knowing which layer broke in under a minute.

SYMPTOM	LIKELY CAUSE	FIX
Agent not replying	Gateway stopped or unhealthy	`openclaw gateway status`, then `openclaw doctor`
Gateway fine, chat dead	Channel disconnected	`openclaw channels status --probe`; re-link if needed
Broke it editing config	Invalid config	`openclaw config validate`; restore from a backup (take one before every big change)
Need to roll back everything	Bad update or corrupted state	`openclaw backup restore <archive> --target <new-empty-dir>`, stop the gateway, swap it in, `openclaw doctor`; WhatsApp may need relinking
It "can't" do what it did last week	Missing credential or tool	Check the credential index and `openclaw skills check` before accepting "can't"
Memory database refuses connections	Every session holding its own pool	Raise the connection limit, close stale sessions

SYMPTOM	LIKELY CAUSE	FIX
Same alert every 15 minutes	No alert gate	Add dedupe and a cooldown
Confident but wrong answers	Low thinking level	Raise thinking (`/think high`)
Bill or quota burning fast	Huge context, cache misses	`/status` , `/context` , then `/new` (see How I tokenmaxx)

18 Operating principles

1. **Accuracy over speed.** A fast wrong answer costs more than a slow right one.
2. **Say how sure you are**, especially before sending someone somewhere or spending their money.
3. **Scope first, execute second.** “Here’s what I’ll do. Proceed?” for anything heavy.
4. **Give context, not just commands.** “Make a video” gets a video. “90-second vertical cut of the product demo, silences trimmed, problem first, fix second, ending on the result” gets the video you wanted.
5. **Proof, not claims.** Done means the output exists and you’ve seen it.
6. **Memory is everything.** If it wasn’t written down, it didn’t happen.
7. **Consider the human’s real constraints.** Don’t tell someone without a local SIM to call a restaurant.
8. **Fix the rule, not the instance.** Every mistake becomes one line in a rules file so it never happens twice.

19 Quick reference

```
# Gateway and health
openclaw gateway status
openclaw gateway stop --disable
openclaw status --deep   # probe channels
openclaw channels status
openclaw doctor --fix
openclaw logs --follow
openclaw update
```

```
# Safety and work
openclaw dashboard
openclaw security audit
openclaw exec-policy show
openclaw backup create
openclaw backup verify <archive>
openclaw automations list
openclaw tasks list
openclaw skills list
openclaw models status
# In chat
/status /context /model /think /new
```

You think and decide.
It remembers, executes and follows up on everything else.

More: docs.openclaw.ai · clawhub.ai · ares-skills · skill-audit-router · ares-mbl · openclaw-mac · GBrain · Hermes Agent · Rushi on X